

Redis Design — b-secure Platform

Production Redis design reference: key naming, data structures, TTL strategy, Django Channels, and operational guide.

Table of Contents

1. [Overview](#)
 2. [Connection Setup](#)
 3. [Key Naming Convention](#)
 4. [Data Structures by Use Case](#)
 5. [TTL Strategy Table](#)
 6. [Django Channels Layer](#)
 7. [Memory Management](#)
 8. [Redis Sentinel / ElastiCache HA](#)
 9. [Monitoring & Debug Commands](#)
-

1. Overview

Redis serves five distinct roles in the b-secure platform:

Role	Redis DB	Library	Description
Cache + Sessions	DB 0	django-redis	Guard profiles, nearby results, Django sessions
Celery Broker	DB 1	kombu	Task queues (high/default/low/scheduled)
Celery Results	DB 1	celery	Task result metadata
Channels Layer	DB 2	channels_redis	WebSocket pub/sub for live tracking
Real-time State	DB 0	redis-py / django-redis	

Role	Redis DB	Library	Description
			Guard location, online status, OTP, rate limits

Using separate DBs provides logical isolation and allows independent FLUSHDB without affecting other subsystems. In production (ElastiCache), all DBs reside on the same cluster node; use separate clusters if strict isolation is required.

2. Connection Setup

2.1 Django Cache Settings (django-redis)

```
# settings/base.py
CACHES = {
    "default": {
        "BACKEND": "django_redis.cache.RedisCache",
        "LOCATION": "redis://redis:6379/0",
        "OPTIONS": {
            "CLIENT_CLASS": "django_redis.client.DefaultClient",
            "CONNECTION_POOL_KWARGS": {
                "max_connections": 50,
                "socket_connect_timeout": 5,
                "socket_timeout": 5,
                "retry_on_timeout": True,
            },
            "COMPRESSOR":
                "django_redis.compressor.zlib.ZlibCompressor",
            "SERIALIZER":
                "django_redis.serializers.json.JSONSerializer",
            "IGNORE_EXCEPTIONS": False,
            # raise on Redis failure (do NOT silence in prod)
            "PASSWORD": env("REDIS_PASSWORD", default=""),
        },
        "KEY_PREFIX": "bsecure",
        "TIMEOUT": 300, # default TTL for cache.set() without
            explicit timeout
    }
}

# Session engine backed by Redis
SESSION_ENGINE = "django.contrib.sessions.backends.cache"
SESSION_CACHE_ALIAS = "default"
SESSION_COOKIE_AGE = 86400 # 1 day in seconds
```

2.2 Direct redis-py Connection (for non-cache operations)

```
# core/redis_client.py
import redis
from django.conf import settings

# DB 0 – cache, sessions, real-time state
_pool_db0 = redis.ConnectionPool.from_url(
    settings.REDIS_URL,          # e.g.
    "redis://:password@redis:6379/0"
    max_connections=50,
    socket_connect_timeout=5,
    socket_timeout=5,
    decode_responses=True,      # return str not bytes
)

# DB 2 – for any direct Channels-layer key inspection
_pool_db2 = redis.ConnectionPool.from_url(
    settings.REDIS_CHANNELS_URL,
    max_connections=20,
    decode_responses=True,
)

def get_redis() -> redis.Redis:
    """Return a Redis client for DB 0 (cache / real-time
    state)."""
    return redis.Redis(connection_pool=_pool_db0)
```

2.3 Environment Variables

```
# .env
REDIS_URL=redis://:yourpassword@redis:6379/0
REDIS_CELERY_URL=redis://:yourpassword@redis:6379/1
REDIS_CHANNELS_URL=redis://:yourpassword@redis:6379/2
```

3. Key Naming Convention

Format: bsecure:{service}:{entity}:{id}[:{field}]

All keys are prefixed with bsecure: to namespace the application within a shared Redis instance. Use colons as separators — never dots or slashes.

Key Pattern	Type	Description
bsecure:location:guard:{guard_id}	Hash	Real-time guard location fields
bsecure:online:guards	Set	

Key Pattern	Type	Description
		SET of currently online guard IDs
bsecure:status:guard:{guard_id}	String	Heartbeat TTL key — exists = online
bsecure:otp:{phone}:{type}	String	Hashed OTP value
bsecure:otp:ratelimit:{phone}	String	OTP request count in window
bsecure:ratelimit:{endpoint}:{user_id}:{window}	String	API rate limit counter
bsecure:cache:guard:{guard_id}	String (JSON)	Cached guard profile
bsecure:cache:user:{user_id}	String (JSON)	Cached user profile
bsecure:cache:nearby:{lat}:{lon}:{radius}	String (JSON)	Cached nearby guards result
bsecure:booking:{booking_id}:state	Hash	Live booking state for WS consumers
bsecure:fcm:user:{user_id}	Hash	FCM push token for user
bsecure:fcm:guard:{guard_id}	Hash	FCM push token for guard
bsecure:leaderboard:guards:rating	Sorted Set	Guards ranked by average rating
bsecure:leaderboard:guards:bookings:{YYYY-MM}	Sorted Set	Guards ranked by monthly bookings
bsecure:celery:dedup:{task}:{key}	String	Task deduplication lock
django.contrib.sessions.cache:{session_key}	String	Django session data

4. Data Structures by Use Case

4a. Guard Real-Time Location (Hash)

Key: bsecure:location:guard:{guard_id}

Type: Hash

TTL: 300 seconds — if not refreshed, guard is considered offline

Field	Type	Example
lat	float string	"12.9716"
lon	float string	"77.5946"
heading	float string	"270.5"
speed	float string	"2.3" (km/h)
updated_at	ISO string	"2025-05-28T10:30:00Z"
booking_id	string	"4891" or ""

```
# tracking/redis_ops.py
from core.redis_client import get_redis
from django.utils import timezone

def update_guard_location(guard_id: int, lat: float, lon: float,
                        heading: float = 0.0, speed: float =
                        0.0, booking_id: int = None):
    r = get_redis()
    key = f"bsecure:location:guard:{guard_id}"

    r.hset(key, mapping={
        "lat": str(lat),
        "lon": str(lon),
        "heading": str(heading),
        "speed": str(speed),
        "updated_at": timezone.now().isoformat(),
        "booking_id": str(booking_id) if booking_id else "",
    })
    r.expire(key, 300) # reset TTL on every update

def get_guard_location(guard_id: int) -> dict | None:
    r = get_redis()
    data = r.hgetall(f"bsecure:location:guard:{guard_id}")
    if not data:
        return None # guard offline or TTL expired
    return {
        "lat": float(data["lat"]),
        "lon": float(data["lon"]),
        "heading": float(data["heading"]),
```

```

    "speed": float(data["speed"]),
    "updated_at": data["updated_at"],
    "booking_id": int(data["booking_id"]) if
    data.get("booking_id") else None,
}

```

4b. Guard Online Status (Set + Heartbeat String)

Key 1: bsecure:online:guards — Redis SET, members = guard IDs (strings)

Key 2: bsecure:status:guard:{guard_id} — STRING, value = "1", TTL = 60s

Heartbeat pattern: guard sends a ping every 30 s. If two consecutive pings are missed (60 s TTL expires), the guard is automatically offline.

```

# tracking/online_status.py
from core.redis_client import get_redis
HEARTBEAT_TTL = 60 # seconds
HEARTBEAT_INTERVAL = 30 # client should ping every 30s

def go_online(guard_id: int):
    r = get_redis()
    r.sadd("bsecure:online:guards", str(guard_id))
    r.set(f"bsecure:status:guard:{guard_id}", "1",
        ex=HEARTBEAT_TTL)

def go_offline(guard_id: int):
    r = get_redis()
    r.srem("bsecure:online:guards", str(guard_id))
    r.delete(f"bsecure:status:guard:{guard_id}")

def heartbeat(guard_id: int):
    """Called by guard app every 30 s to reset the TTL."""
    r = get_redis()
    # Only refresh if they were already online (SET member check)
    if r.sismember("bsecure:online:guards", str(guard_id)):
        r.set(f"bsecure:status:guard:{guard_id}", "1",
            ex=HEARTBEAT_TTL)
    else:
        go_online(guard_id) # re-register if SET was cleared

def is_online(guard_id: int) -> bool:
    r = get_redis()
    return bool(r.exists(f"bsecure:status:guard:{guard_id}"))

def get_online_guard_ids() -> set[int]:

```

```

r = get_redis()
return {int(gid) for gid in
        r.smembers("bsecure:online:guards")}

```

Sync job: A Celery beat task (sync_guard_online_status, every 1 minute) reconciles the bsecure:online:guards SET against the heartbeat keys — removes any guard from the SET whose heartbeat key has expired. This prevents stale entries in the SET outliving their TTL keys.

4c. OTP Verification (String)

Key: bsecure:otp:{phone}:{type} — value = bcrypt hash of OTP

TTL: 300 seconds (5 minutes)

Rate limit key: bsecure:otp:ratelimit:{phone} — increment counter, TTL = 3600s

```

# auth/otp.py
import bcrypt
import random
import string
from core.redis_client import get_redis

OTP_TTL = 300      # 5 minutes
RATE_LIMIT_MAX = 5 # max 5 OTPs per hour per phone
RATE_LIMIT_TTL = 3600 # 1 hour window

def generate_and_store_otp(phone: str, otp_type: str = "login")
    -> str:
    r = get_redis()

    # Rate limiting
    rate_key = f"bsecure:otp:ratelimit:{phone}"
    count = r.incr(rate_key)
    if count == 1:
        r.expire(rate_key, RATE_LIMIT_TTL)
    if count > RATE_LIMIT_MAX:
        raise
        PermissionError("Too many OTP requests. Try again in 1
        hour.")

    # Generate 6-digit OTP
    otp = "".join(random.choices(string.digits, k=6))
    hashed = bcrypt.hashpw(otp.encode(),
        bcrypt.gensalt()).decode()

    otp_key = f"bsecure:otp:{phone}:{otp_type}"
    r.set(otp_key, hashed, ex=OTP_TTL)
    return otp # send via SMS — do NOT store plaintext

```

```

def verify_otp(phone: str, otp: str, otp_type: str = "login") ->
    bool:
    r = get_redis()
    otp_key = f"bsecure:otp:{phone}:{otp_type}"
    stored_hash = r.get(otp_key)

    if not stored_hash:
        return False # expired or never issued

    valid = bcrypt.checkpw(otp.encode(), stored_hash.encode())
    if valid:
        r.delete(otp_key) # one-time use – delete on success
    return valid

```

4d. API Rate Limiting (Fixed Window)

Key: bsecure:ratelimit:{endpoint}:{user_id}:{window}
 {window} = Unix timestamp floored to window size (e.g., 60s =
 int(time.time() // 60))

```

# core/rate_limit.py
import time
from functools import wraps
from core.redis_client import get_redis
from rest_framework.response import Response

def rate_limit(max_requests: int, window_seconds: int, scope: str
               = "api"):
    """
    Decorator for DRF APIView methods.
    Uses a fixed-window counter in Redis.
    """
    def decorator(view_func):
        @wraps(view_func)
        def wrapper(self, request, *args, **kwargs):
            r = get_redis()
            user_id = request.user.id if
            request.user.is_authenticated else
            request.META.get("REMOTE_ADDR")
            window = int(time.time() // window_seconds)
            key = f"bsecure:ratelimit:{scope}:{user_id}:
            {window}"

            count = r.incr(key)
            if count == 1:
                r.expire(key, window_seconds + 5)
                # small buffer for clock skew

            if count > max_requests:
                return Response(
                    {"detail": f"Rate limit exceeded. Max
                    {max_requests} requests per {window_seconds}s."},

```



```

        status=429,
        headers={"Retry-After": str(window_seconds)},
    )
    return view_func(self, request, *args, **kwargs)
return wrapper
return decorator

```

Usage on a view

```

class NearbyGuardsView(APIView):
    @rate_limit(max_requests=30, window_seconds=60,
                scope="nearby_guards")
    def get(self, request):
        ...

```

4e. Session Store

settings/base.py – already shown in section 2, repeated for completeness

```

SESSION_ENGINE = "django.contrib.sessions.backends.cache"
SESSION_CACHE_ALIAS = "default"
SESSION_COOKIE_AGE = 86400 # 1 day
SESSION_COOKIE_SECURE = True # HTTPS only
SESSION_COOKIE_HTTPONLY = True
SESSION_COOKIE_SAMESITE = "Lax"

```

Django automatically manages session keys. The key pattern in Redis is:

```
django.contrib.sessions.cache:{session_key}
```

The value is the serialised (JSON + zlib compressed) session dictionary. TTL = SESSION_COOKIE_AGE.

4f. Django Cache — Per-Object and Per-View

guards/cache.py

```

from django.core.cache import cache
import json

```

```

GUARD_PROFILE_TTL = 300 # 5 minutes
NEARBY_RESULT_TTL = 30 # 30 seconds (location data changes fast)
USER_PROFILE_TTL = 300

```

```

def get_guard_profile_cache(guard_id: int):
    key = f"bsecure:cache:guard:{guard_id}"
    return cache.get(key)

```

```

def set_guard_profile_cache(guard_id: int, data: dict):
    key = f"bsecure:cache:guard:{guard_id}"

```

```

cache.set(key, data, timeout=GUARD_PROFILE_TTL)

def invalidate_guard_profile_cache(guard_id: int):
    cache.delete(f"bsecure:cache:guard:{guard_id}")

def get_nearby_cache_key(lat: float, lon: float, radius: int) ->
    str:
    # Round coordinates to 3 decimal places (~111m precision) to
    improve cache hit rate
    return f"bsecure:cache:nearby:{round(lat, 3)}:{round(lon, 3)}:
        {radius}"

# Signal-based invalidation
from django.db.models.signals import post_save
from django.dispatch import receiver
from guards.models import Guard

@receiver(post_save, sender=Guard)
def invalidate_guard_cache(sender, instance, **kwargs):
    invalidate_guard_profile_cache(instance.id)

# DRF view with manual caching
class GuardDetailView(APIView):
    def get(self, request, guard_id: int):
        cached = get_guard_profile_cache(guard_id)
        if cached:
            return Response(cached)

        guard = get_object_or_404(Guard, id=guard_id,
            is_active=True)
        data = GuardDetailSerializer(guard).data
        set_guard_profile_cache(guard_id, data)
        return Response(data)

```

4g. Booking State Cache (Hash)

Key: bsecure:booking:{booking_id}:state

Type: Hash

TTL: 86400 seconds (24 hours)

Used by Django Channels WebSocket consumers to read booking state without a DB query on every location update message.

Field	Example
status	"active"
guard_id	"42"
user_id	"101"

Field	Example
started_at	"2025-05-28T09:00:00Z"
scheduled_at	"2025-05-28T09:00:00Z"

```
# bookings/state_cache.py
from core.redis_client import get_redis

def cache_booking_state(booking):
    r = get_redis()
    key = f"bsecure:booking:{booking.id}:state"
    r.hset(key, mapping={
        "status": booking.status,
        "guard_id": str(booking.guard_id),
        "user_id": str(booking.user_id),
        "started_at": booking.started_at.isoformat() if
            booking.started_at else "",
        "scheduled_at": booking.scheduled_at.isoformat(),
    })
    r.expire(key, 86400)

def get_booking_state(booking_id: int) -> dict | None:
    r = get_redis()
    data = r.hgetall(f"bsecure:booking:{booking_id}:state")
    return data if data else None

def invalidate_booking_state(booking_id: int):
    get_redis().delete(f"bsecure:booking:{booking_id}:state")
```

4h. Push Notification Token Store (Hash)

Key: bsecure:fcm:user:{user_id} or bsecure:fcm:guard:{guard_id}

Field	Example
token	"dAM3...FCM_TOKEN..."
platform	"android" or "ios"
updated_at	"2025-05-28T10:00:00Z"

```
# notifications/token_store.py
from core.redis_client import get_redis
from django.utils import timezone

def store_fcm_token(entity_type: str, entity_id: int, token: str,
    platform: str):
    """entity_type: 'user' or 'guard'"""
    r = get_redis()
```

```

key = f"bsecure:fc:{entity_type}:{entity_id}"
r.hset(key, mapping={
    "token": token,
    "platform": platform,
    "updated_at": timezone.now().isoformat(),
})

# No TTL – tokens persist until explicitly overwritten or
user logs out

```

```

def get_fcm_token(entity_type: str, entity_id: int) -> str | None:
    r = get_redis()
    data = r.hgetall(f"bsecure:fc:{entity_type}:{entity_id}")
    return data.get("token") if data else None

```

```

def delete_fcm_token(entity_type: str, entity_id: int):
    get_redis().delete(f"bsecure:fc:{entity_type}:{entity_id}")

```

4i. Leaderboard / Guard Rankings (Sorted Set)

Key: bsecure:leaderboard:guards:rating

Type: Sorted Set — member = guard_id (string), score = average_rating (float)

```

# leaderboard/redis_ops.py
from core.redis_client import get_redis

```

```

RATING_LEADERBOARD_KEY = "bsecure:leaderboard:guards:rating"

```

```

def update_guard_rating_score(guard_id: int, average_rating:
    float):
    r = get_redis()
    r.zadd(RATING_LEADERBOARD_KEY, {str(guard_id):
        average_rating})

```

```

def get_top_guards_by_rating(n: int = 10) -> list[tuple[str,
    float]]:
    """Returns [(guard_id_str, score), ...] descending by
    rating."""
    r = get_redis()
    return r.zrevrange(RATING_LEADERBOARD_KEY, 0, n - 1,
        withscores=True)

```

```

def get_guard_rating_rank(guard_id: int) -> int | None:
    """Returns 0-based rank (0 = highest rated). None if not in
    leaderboard."""
    r = get_redis()
    rank = r.zrevrank(RATING_LEADERBOARD_KEY, str(guard_id))

```

```

    return rank    # None if member not found

# Monthly booking leaderboard
def get_monthly_bookings_key(year: int, month: int) -> str:
    return f"bsecure:leaderboard:guards:bookings:{year:04d}-{month:02d}"

def increment_guard_booking_count(guard_id: int, year: int, month: int):
    r = get_redis()
    key = get_monthly_bookings_key(year, month)
    r.zincrby(key, 1, str(guard_id))
    # Expire monthly leaderboard 90 days after month end
    r.expire(key, 90 * 86400)

```

5. TTL Strategy Table

Key Pattern	TTL	Eviction Note
bsecure:location:guard:{id}	300s	Guard offline if expired — no eviction needed
bsecure:status:guard:{id}	60s	Heartbeat key — expires naturally
bsecure:online:guards	No TTL	Managed by sync Celery task
bsecure:otp:{phone}:{type}	300s	OTP expires in 5 minutes
bsecure:otp:ratelimit:{phone}	3600s	Rate limit window: 1 hour
bsecure:ratelimit:{...}	window+5s	Fixed window; expires automatically
bsecure:cache:guard:{id}	300s	Cache: invalidated on model save
bsecure:cache:user:{id}	300s	Cache: invalidated on model save
bsecure:cache:nearby:{...}	30s	Short TTL — guard positions change
bsecure:booking:{id}:state	86400s	24h; deleted on booking terminal state
bsecure:fcm:{type}:{id}	No TTL	Deleted on logout / token refresh
bsecure:leaderboard:guards:rating	No TTL	

Key Pattern	TTL	Eviction Note
		Updated by Celery beat every 30 min
bsecure:leaderboard:guards:bookings:{month}	90 days	Auto-expires old month data
bsecure:celery:dedup:{...}	Task window	Task-specific (seconds to minutes)
django.contrib.sessions.cache:{key}	86400s	SESSION_COOKIE_AGE

Eviction policy per DB:

```
# redis.conf (or ElastiCache parameter group)
# DB 0 (cache + real-time): evict least recently used cache keys
first
maxmemory-policy allkeys-lru

# DB 1 (Celery broker): NEVER evict tasks
maxmemory-policy noeviction

# DB 2 (Channels): evict LRU is acceptable for channel layer
maxmemory-policy allkeys-lru
```

6. Django Channels Layer

6.1 Configuration

```
# settings/base.py
CHANNEL_LAYERS = {
    "default": {
        "BACKEND": "channels_redis.core.RedisChannelLayer",
        "CONFIG": {
            "hosts": [("redis", 6379)],
            "password": env("REDIS_PASSWORD", default=""),
            "db": 2,
            "capacity": 1500,
            # max messages per channel before backpressure
            "expiry": 10, # message TTL in seconds
            "group_expiry": 86400, # group membership TTL
            # (24h)
            "channel_capacity": {
                "http.request": 200,
                "websocket.send*": 20,
            },
        },
    },
}
```

6.2 Channel Group Names

Group Name	Members	Purpose
booking_{booking_id}	User + Guard consumer	Live location updates during booking
guard_{guard_id}	Guard's own consumer	Incoming booking requests, admin messages
user_{user_id}	User's own consumer	Booking status changes, notifications
admin_dashboard	Admin consumers	Platform-wide events, SOS alerts
sos_{sos_id}	User + Guard + Admin	SOS event coordination

```
# tracking/consumers.py (excerpt)
from channels.generic.websocket import AsyncJsonWebSocketConsumer
from channels.db import database_sync_to_async

class BookingTrackingConsumer(AsyncJsonWebSocketConsumer):
    async def connect(self):
        self.booking_id = self.scope["url_route"]["kwargs"]
            ["booking_id"]
        self.group_name = f"booking_{self.booking_id}"

        # Join the booking group
        await self.channel_layer.group_add(self.group_name,
            self.channel_name)
        await self.accept()

    async def disconnect(self, close_code):
        await self.channel_layer.group_discard(self.group_name,
            self.channel_name)

    # Called when guard sends location update
    async def receive_json(self, content):
        if content.get("type") == "location_update":
            await self.channel_layer.group_send(
                self.group_name,
                {
                    "type": "guard.location", # maps to
guard_location() handler
                    "lat": content["lat"],
                    "lon": content["lon"],
                    "heading": content.get("heading", 0),
                }
            )

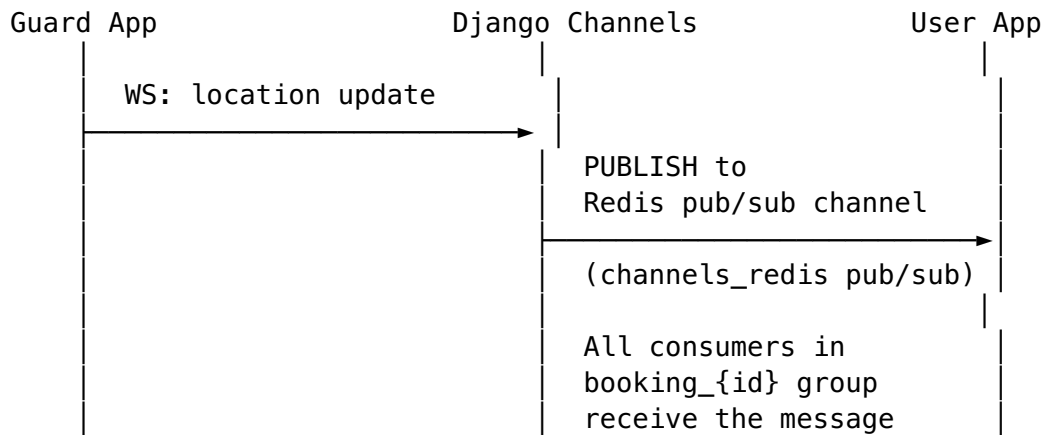
    async def guard_location(self, event):
```

```

        """Broadcast location to all consumers in the group (user
        + guard)."""
        await self.send_json(event)

```

6.3 How Channels Uses Redis Internally



Redis Sorted Sets store group members. Messages are published to a Redis channel named after the internal channel name; all Channels layer nodes subscribe and fan out to local WebSocket connections.

7. Memory Management

7.1 Estimating Memory Usage

Formula: $\text{memory} \approx \text{key_count} \times \text{avg_value_size} \times \text{overhead_factor}$ (1.5)

Example for DB 0:

- 500 guards × location hash (6 fields × ~30 bytes) = ~90 KB
 - 500 guards × status string = ~5 KB
 - 5000 guard profile caches × ~2 KB JSON = ~10 MB
 - 10000 session keys × ~500 bytes = ~5 MB
 - 2000 booking state hashes × ~200 bytes = ~400 KB
- Total estimate: ~16 MB (well within a t3.small ElastiCache node)

7.2 Monitoring Memory

```

# Connect to Redis
redis-cli -h redis -p 6379 -a "$REDIS_PASSWORD"

```

```

# Overall memory stats
INFO memory

```

```

# Number of keys per DB
INFO keyspace

```



```
# Size of a specific key
MEMORY USAGE bsecure:cache:guard:42

# Largest keys (top 10)
redis-cli --bigkeys

# Count keys matching a pattern (use SCAN, never KEYS in
# production)
redis-cli --scan --pattern 'bsecure:cache:guard:*' | wc -l
```

7.3 Memory Alerts (CloudWatch / Prometheus)

```
# Recommended thresholds for a 1 GB ElastiCache node
used_memory > 700 MB    → WARNING (70% utilisation)
used_memory > 900 MB    → CRITICAL (90% utilisation – risk of
    eviction)
evicted_keys > 0        → WARNING (LRU eviction has started)
```

8. Redis Sentinel / ElastiCache HA

8.1 ElastiCache (Recommended for AWS)

```
# settings/production.py
# ElastiCache Redis with automatic failover
# cluster mode DISABLED (single shard) – simpler for this workload

CACHES = {
    "default": {
        "BACKEND": "django_redis.cache.RedisCache",
        "LOCATION": f"rediss://
{env('ELASTICACHE_ENDPOINT')}:6379/0",
        # 'rediss://' = TLS (required for ElastiCache in-transit
        # encryption)
        "OPTIONS": {
            "CLIENT_CLASS": "django_redis.client.DefaultClient",
            "CONNECTION_POOL_KWARGS": {
                "max_connections": 50,
                "ssl_cert_reqs": None, # ElastiCache uses self-
                signed cert
            },
            "PASSWORD": env("REDIS_AUTH_TOKEN"),
        },
    },
}
```

8.2 Redis Sentinel (Self-Hosted)

```
# For self-hosted Redis with Sentinel
from django_redis.client import import SentinelClient

CACHES = {
    "default": {
        "BACKEND": "django_redis.cache.RedisCache",
        "LOCATION": "redis://mymaster/0",
        "OPTIONS": {
            "CLIENT_CLASS": "django_redis.client.SentinelClient",
            "SENTINELS": [
                ("sentinel-1", 26379),
                ("sentinel-2", 26379),
                ("sentinel-3", 26379),
            ],
            "SENTINEL_SERVICE_NAME": "mymaster",
            "PASSWORD": env("REDIS_PASSWORD"),
            "SENTINEL_KWARGS": {"password":
env("REDIS_SENTINEL_PASSWORD")}},
        },
    }
}
```

9. Monitoring & Debug Commands

```
#  MONITOR streams ALL commands – extreme I/O overhead, dev-
only
redis-cli MONITOR

# Scan all b-secure keys (safe, non-blocking, use SCAN not KEYS)
redis-cli --scan --pattern 'bsecure:*'

# Slow query log (last 10 slow commands)
redis-cli SLOWLOG GET 10
redis-cli SLOWLOG RESET # clear after review

# Queue depth for Celery queues
redis-cli LLEN _kombu.binding.high_priority
redis-cli LLEN _kombu.binding.default
redis-cli LLEN _kombu.binding.low_priority

# TTL remaining on a key
redis-cli TTL bsecure:status:guard:42

# All fields of a guard location hash
redis-cli HGETALL bsecure:location:guard:42

# Top 5 guards by rating leaderboard
```

```
redis-cli ZREVRANGE bsecure:leaderboard:guards:rating 0 4
WITHSCORES
```

Count keys by pattern using SCAN + Lua

```
redis-cli --eval - <<'EOF'
local count = 0
local cursor = "0"
repeat
    local result = redis.call("SCAN", cursor, "MATCH",
                              "bsecure:cache:guard:*", "COUNT", 100)
    cursor = result[1]
    count = count + #result[2]
until cursor == "0"
return count
EOF
```

Command	Safe in Prod?	Purpose
SCAN	✓	Iterate keys without blocking
KEYS *	✗	Blocks server — never in production
MONITOR	⚠ Dev only	Full command stream — 50% perf hit
SLOWLOG GET	✓	Diagnose slow commands
INFO all	✓	Full server statistics
DBSIZE	✓	Key count for current DB
FLUSHDB	✗ Destructive	Clears current DB — only in emergencies
MEMORY USAGE key	✓	Per-key memory size
DEBUG SLEEP	✗	Testing only — blocks server